Практика по Stream API

Юрий Литвинов
y.litvinov@spbu.ru

24.10.2026

Stream API

- Работа с последовательностями элементов в функциональном стиле
  - Появились в Java 8
- Поток (Stream) — абстракция конвейера обработки данных (не контейнер)
- Источник: list.stream(), Arrays.stream(...), Files.lines(...), Stream.of(...), ...
- Промежуточные операции: filter, map, sorted, distinct, ...
  - Ленивые, т.е. ничего не делают, пока не попросят следующий элемент
- Терминальные операции: collect, forEach, count, findFirst, ...
  - Выполняют операцию, возвращают результат (или побочный эффект)
  - Забирают данные из потока, так что повторно использовать его нельзя
    - Контейнер, на базе которого поток создан, не меняют

Пример

```
var activeUsersList = list.stream()
    .filter(User::isActive)
    .map(User::getName)
    .toList();
```

Популярные операции

- filter — принимает предикат, возвращает поток с элементами, удовлетворяющими предикату
- map — применяет функцию к каждому элементу потока, возвращает поток результатов
  - flatMap — если результат функции — поток, склеивает потоки-результаты в один большой («уплощает» коллекцию)
- collect — группирует (или агрегирует) данные в потоке
  - Collectors — фабрика популярных коллекторов, например, Collectors.groupingBy и Collectors.summingInt
- sorted, distinct — сортирует, выкидывает повторы
- Ещё есть parallelStream — возвращает поток, в котором операции сами выполняются параллельно
  - Можно прострелить себе ногу ошибками синхронизации, плохо работают на небольших коллекциях

Более развёрнутый пример

Без Stream API

```
List<String> result = new ArrayList<>();
for (User u : users) {
    if (u.isActive() && u.getAge() >= 18) {
        result.add(u.getName());
    }
}
result.sort(Comparator.comparing(User::getName));
```

С Stream API:

```
users.stream()
    .filter(User::isActive)
    .filter(u -> u.getAge() >= 18)
    .sorted(Comparator.comparing(User::getName))
    .map(User::getName)
    .toList(); // До Java 16 — collect(Collectors.toList())
```

Задача на пару

Реализовать с помощью Java Stream API:

- Методы в FirstPartTasks и SecondPartTasks
- Тесты
  - Тесты в FirstPartTasksTest должны проходить как есть
  - Тесты в SecondPartTasksTest надо написать самостоятельно
- Писать в максимально функциональном стиле
- Зависимости для тестов:
  - JUnit
  - Guava